

# What is exposed, and what to do about it

You asked whether someone could download our catalogue and walk off with the embeddings, the tagging, and the rest of the work the paid features are built on. The short answer is yes. The useful answer is that most of it was never as protectable as it looked, one part of it genuinely is, and the two most urgent problems are not really security problems at all.

## 4 requests

is all it takes to download the entire artwork tagging corpus, permanently and offline, using a key that ships inside our own website.

## 1 line

keeps the tagging out of a file we publish publicly every day, and that line is not saved into version control.

## 34 MB

of raw artwork “fingerprints” already sit in a folder anyone can download with no key at all. On purpose. The phone scanner needs them.

## 2 days

of careful work fixes the part that is worth fixing. Not the three weeks, and not the new database.

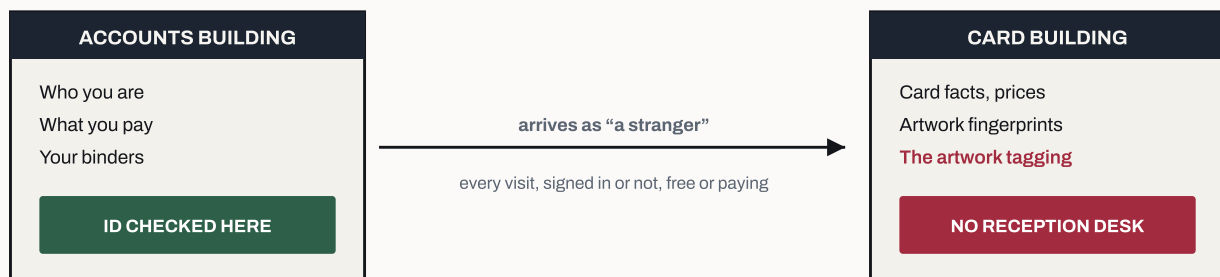
# 1 THE SETUP

## Two buildings, and only one of them checks ID

Everything rests on one fact about how the apps are built, so it is worth 60 seconds.

We run two separate databases. Think of them as two buildings. The **Accounts building** holds people: who signed up, who is paying, who is on a trial. The **Card building** holds everything about Pokémon cards: names, prices, pictures, and all the clever derived data our features are built on.

When someone opens michi maker or TCGScan, they sign in at the Accounts building. Then, to look at cards, the app walks over to the Card building. **And it walks in without showing any ID.** Not because of a bug, but because that was never wired up. The Card building has no reception desk.



Because no identity ever reaches the Card building, it cannot tell a paying member from a stranger. Every protection there has to be either open to all or closed to all. That single fact explains almost every finding in this report.

### The key that ships in our own website

To read the Card building at all, the app carries a key. This key is *designed* to be public: it is inside our website's code and inside the phone app, so anybody who wants it has it in about a minute. That is normal and fine. The catch is that a public key opens whatever the building decides to leave unlocked, and nobody had ever gone through and decided.

#### THE MISTAKE THAT WAS WAITING TO HAPPEN

When you add a new column of data to a table, it is readable by the public *by default*. Nobody has to grant anything. So every time we

added something valuable over the past two months, we published it, and nothing anywhere said so.

## What is actually behind each door

I checked all of it, by actually downloading things as an anonymous stranger would. Here is the honest picture. The important thing to notice is that the doors are open for *three different reasons*, and only one of them is a problem you can fix.

■ Open on purpose, and has to stay open

■ Open by accident. This is the fixable part.

|                                                                                                                                                |             |
|------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>Artwork fingerprints (“embeddings”)</b><br>34 MB file, 24,720 cards. The phone scanner downloads it to work offline.                        | PUBLIC FILE |
| <b>Colour data for every card</b><br>658 KB file, 27,415 cards. Makes colour search instant instead of slow.                                   | PUBLIC FILE |
| <b>The trained recognition model itself</b><br>The phone downloads it to scan cards. Cannot be hidden from someone running the app.            | PUBLIC FILE |
| <b>The artwork tagging — 3,272 cards</b><br>Every scene description and every theme word. This IS theme search. Four requests takes all of it. | FIX THIS    |
| <b>Control switches a stranger can flip</b><br>Eight to ten of them, including the one that decides which scanner model every phone uses.      | FIX TODAY   |
| <b>Every daily price we have ever recorded</b><br>The one thing here nobody can recreate. Our own notes told the team to leave it open.        | DECIDE      |
| <b>The scanner’s “cards that confuse it” list</b><br>3,669 rows of which cards look alike and why. Hard-won, and open.                         | CHEAP FIX   |

Measured by fetching each one as an anonymous visitor. The three amber rows are not mistakes; they are the cost of features we sell. The four red rows are the report.

### Why the fingerprints and colours cannot be locked away

This is the part that surprised me, and it reverses the advice I gave earlier in the week. Those files have to be downloadable, because the phone scanner works *without internet*. It cannot ask permission at the moment you point the camera at a card. Locking them means breaking the feature people actually pay for.

And they are worth less than they sound. An artwork fingerprint is what you get by running a standard, freely available image model over card pictures that *we publish ourselves*. A competent engineer with a rented graphics card recreates all of them in an afternoon. Spending weeks hiding something reproducible for a few hundred dollars is the worst trade on the table.

SO THE WHOLE JOB SHRINKS TO THIS

Close the artwork tagging, close the switches a stranger should never touch, and make a decision about the price history. Everything else is either already public for a good reason, or not worth defending.

### 3

URGENT

## The two things that matter more than any of it

---

Both of these came out of the deep audit rather than the original question, and both are housekeeping rather than security. That is exactly why they are dangerous: nobody was looking.

### One: the tap that holds the tagging back is not screwed down

Once a day, unattended, a job on your machine rebuilds the card catalogue and publishes it for every app to download. For a while that published file *included* the artwork tagging. Someone turned that off with a single line of code, and the apps already know how to read the tagging out of that file and do theme search on the phone, free and unlimited. That code is shipped and waiting.

The problem is that the line turning it off **only exists on your hard drive**. It was never saved into version control. So a routine tidy-up, a fresh copy of the project on a new machine, or any assistant reverting a file puts the tagging back into the public download, and within a day it is permanently in the hands of every registered user. There is a comment in the code claiming this is reversible. It is not, for anyone who downloaded during the window.



Fix: save the file, and move the switch somewhere the daily job cannot flip it. Ten minutes.

Three of your four active project folders have unsaved changes in files that matter this much: 63 in the data project, 33 in the phone app, 6 in the pipeline, 4 in the shared browse code. The michi maker project is the only clean one.

### Two: the written record of the database does not match the database

Every change to the Card building is supposed to be recorded as a numbered file, so anyone can read the history and know what is there. I found two such changes that were applied to the live database but filed in a *different project folder*, with numbers that clash with existing ones. They create a table and three functions that appear nowhere in the official history, and the phone app calls one of them in production right now.

Why that matters for this job: every conclusion of the form “I read all the records, therefore only these two things touch this data” was drawn

from a set of files that demonstrably does not describe reality. Mine included. Before anyone changes a permission, we should ask the live database what is actually in it, which takes about two hours.

# 4

## THE FIX

### A guest list, not a new building

The tagging sits in the same table as ordinary card facts like name and rarity. Today the rule is “strangers may read this table,” which means all fifty columns of it. The fix is to change the rule to name the columns instead: **these forty-seven yes, these three no.**

#### TODAY



“Strangers may read this table” — so all fifty columns, including the three that matter.

#### AFTER



“Strangers may read these forty-seven columns” — the scene description, the theme words and the combined version of both are simply not on the list. Our own search still reads them, on our behalf.

Nothing moves, nothing is deleted, no app is released. The search feature keeps working because it is given permission to read on the visitor’s behalf, the way a librarian can fetch a book from a closed stack.

#### THREE TRAPS THAT WERE FOUND THE HARD WAY

The obvious version of this fix does not work, and the first version I sent to the data team would have appeared to succeed while protecting nothing. Saying “except these three” on top of “all of them” is ignored by the database; you have to withdraw the blanket permission and write the list. A second setting has to be flipped at the same time or card browsing goes blank for every user of both apps. And the tidy one-line way of upgrading the search function silently undoes itself the next time anyone edits it, which has happened seven times in the last two months.

# 5

## YOUR QUESTION

### You offered a second database. I would not take it

You said you were open to duplicating the database for safe testing and to redesigning the data architecture. That is a generous offer and it is aimed at a threat smaller than the offer. Three reasons, plainly.

#### It cannot answer the question

The single setting that decides half of what is exposed lives only in a web dashboard, not in the project files. A copied database would not carry it, so it would give you a clean bill of health on exactly the thing you copied it to check.

#### A cheaper test is better

What needs checking is permissions. You can rehearse the entire change on the real database inside a transaction that is thrown away at the end, against real permissions and real data. A small copy tests them against a pretend version.

#### It is only true for a day

58,686 cards, six model bundles, a 34 MB file and the whole price history. A copy starts drifting the moment it is made.

What I would spend instead is nearly nothing: half a day giving the database a proper direct connection, so changes can be applied as one all-or-nothing batch that can be undone. Today they are pasted into a web page one at a time, which is why this project keeps hitting problems that look fine when applied and break later. That half day is the most valuable item in this whole report, and it has nothing to do with security.

#### EFFORT, AND WHAT IT BUYS

|                                                                                     |                   |                    |                                                                             |
|-------------------------------------------------------------------------------------|-------------------|--------------------|-----------------------------------------------------------------------------|
|  | <b>2 days</b>     | <b>RECOMMENDED</b> | Closes the tagging and the switches. Fixes the record keeping.              |
|  | <b>half a day</b> |                    | A real connection to the database. Worth it on its own merits.              |
|  | <b>3 weeks</b>    |                    | Move the data. Protects nothing more, and can fail silently.                |
|  | <b>6 weeks</b>    |                    | Lock the files. Barrier is one free sign up, and it risks offline scanning. |

The two long options were designed, judged and rejected on the evidence, not skipped. The three-week option ends with a pipeline that can stop publishing colour data for weeks while reporting success twice a day.

#### THE THING YOU SHOULD PROBABLY FIND MOST ANNOYING

Theme search, the feature all of this protects, is not listed in our pricing anywhere. It is not in the plan comparison, not in the payment setup, and the phone app hides it from every tier including VIP. Meanwhile the daily price history, the only thing here that nobody

could ever recreate, is wide open because our own notes told the team to leave it alone.

# 6

OVER TO YOU

## Seven decisions, with a recommendation on each

---

Nothing has been done. The Card building has not been touched and the data team has not been sent this. These are yours to call, roughly in the order I would do them.

- 1 Save the unsaved work, and screw down the tap.** Ten minutes. This is the one that could hand away the tagging permanently while nobody is looking. **Recommend: yes, today.**
- 2 Close the switches a stranger can flip.** One hour. One of them decides which scanner model every phone in the field uses. **Recommend: yes, today.**
- 3 Give the database a proper connection.** Half a day, no risk, and it makes every future change reversible. **Recommend: yes, before touching permissions.**
- 4 Ask the live database what is really in it,** rather than trusting the written record. Two hours. **Recommend: yes.**
- 5 Make the guest list change.** About two days including the rehearsal, one all-or-nothing batch, rehearsed and reversible. **Recommend: yes, once 1 to 4 are done.**
- 6 Decide about the price history.** It is the only asset nobody can reproduce. It should be decided deliberately rather than inherited from an old note. **Recommend: decide it in the same sitting as 5.**
- 7 Check whether this has already happened.** Nobody has looked at the access logs for someone pulling the tagging in bulk. The logs do not keep forever, so this has a deadline. **Recommend: yes, and soon.**

### A second database?

Not for this. Revisit it, at roughly nothing per month, the first time a change has to rebuild one of the shared views, or to prove the project can be built from scratch, which nobody has ever tested.

**Recommend: no**

### Lock the downloadable files?

Not now. Two parts of that work pay for themselves anyway: caching the files on the phone instead of refetching 40 MB, and shrinking the big one from 34 MB to about 3 MB. Do those for speed.

**Recommend: no, but take the speed wins**

## The honest summary

This work buys time and inconvenience, not secrecy. Someone determined, with a vision model and the card pictures we publish ourselves, could rebuild an approximation of the tagging for a few hundred dollars. What is genuinely hard to copy was never in either database and was never at risk: the way the tagging was designed, the theme vocabulary inside michi maker, the evaluation set, and the fact that people keep uploading binders. Two days is the right amount to spend on the part that can actually be closed. Three weeks would not have bought more protection.

Full technical version, with every file reference and the exact SQL: **docs/DATA-EXPOSURE.md** in the michi maker project. Sources for every number in this report are cited there.